

Practice Set — Solutions

The practice sheet contains five Python-for-data-analysis questions: loading data, missing values/duplicates, grouping revenue, sorting, and correlation.

Dataset used

The supplied Excel dataset has 200 rows and 10 columns. It contains order, customer, product, quantity, price, revenue, and region fields. The answers below use this dataset where a real result is useful.

1. Load a CSV file using Pandas and display basic info

Answer:

```
import pandas as pd

df = pd.read_csv("sales.csv")

print(df.head())
print(df.info())
print(df.shape)
print(df.describe(include="all"))
```

For the supplied Excel file, use `read_excel` instead:

```
df = pd.read_excel("SQL_Sales_Dataset_200_Rows.xlsx")
print(df.info())
```

Result for the supplied dataset: 200 rows, 10 columns; all 200 rows are populated in every column. The numeric fields are `order_id`, `quantity`, `unit_price`, and `total_price`.

2. Handle missing values and duplicates using Pandas

```
# Check missing values
print(df.isna().sum())

# Remove duplicate rows
df = df.drop_duplicates()

# Example: fill missing numeric values with the median
# df["unit_price"] = df["unit_price"].fillna(df["unit_price"].median())

# Example: fill missing category values
# df["category"] = df["category"].fillna("Unknown")
```

Actual result: the supplied dataset has 0 missing values in every column and 0 duplicate rows, so no cleaning was required.

3. Group data by category and find total revenue

```
revenue_by_category = (
    df.groupby("category", as_index=False)["total_price"]
      .sum()
      .sort_values("total_price", ascending=False)
)

print(revenue_by_category)
```

Category	Total revenue
Furniture	714,399
Grocery	672,147
Electronics	549,302
Clothing	484,259

Interpretation: `total_price` is used as the revenue field in the supplied dataset.

4. Sort data by multiple columns using Python

```
sorted_df = df.sort_values(  
    by=["region", "total_price"],  
    ascending=[True, False]  
)  
  
print(sorted_df[["region", "category", "total_price"]].head(10))
```

This sorts region in ascending order and, within each region, total_price in descending order.

5. Create a correlation matrix for numerical columns

```
numeric_df = df.select_dtypes(include="number")  
correlation_matrix = numeric_df.corr()  
  
print(correlation_matrix.round(3))
```

	order_id	quantity	unit_price	total_price
order_id	1.000	-0.018	0.102	0.099
quantity	-0.018	1.000	0.048	0.655
unit_price	0.102	0.048	1.000	0.671
total_price	0.099	0.655	0.671	1.000

The matrix shows pairwise Pearson correlations. In this dataset, quantity and total_price have correlation 0.655, while unit_price and total_price have correlation 0.671. Correlation describes association; it does not by itself establish causation.

Quick revision summary

1) read_csv/read_excel loads data. 2) isna().sum() checks missing values and drop_duplicates() removes exact duplicate rows. 3) groupby(...).sum() aggregates revenue. 4) sort_values() can use multiple columns with matching ascending flags. 5) corr() creates a numerical correlation matrix.